

Instruction Granularity as Planning Width

Formalizing and Benchmarking for Vision-Language-Action Models



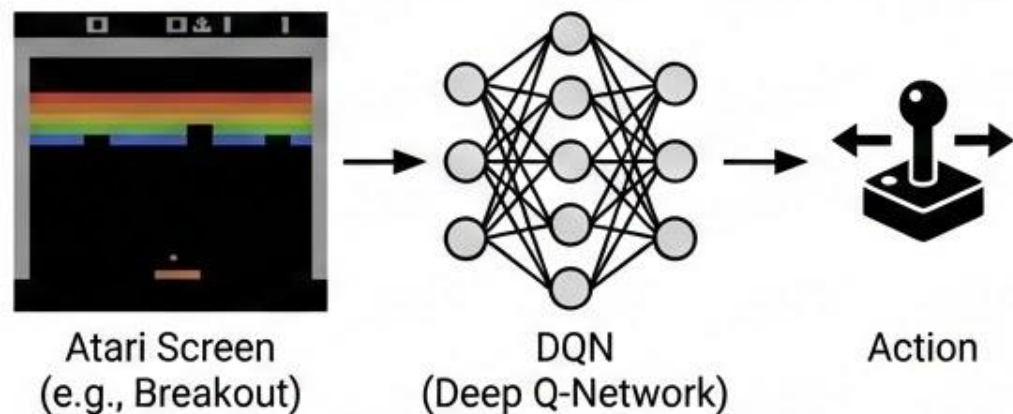
IJCAI Submission

Sukai Huang

NLP reading group presentation

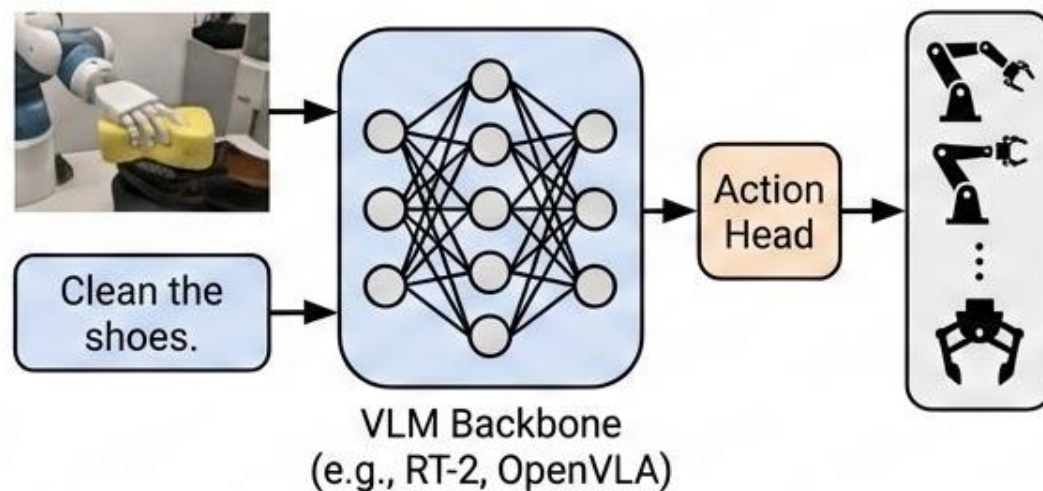
Background: From Deep RL to VLA

Past: Deep RL (VA - Vision-Action)



Example: DQN on Atari games. Input is only Vision (V), output is Action (A).

Present: VLA (Vision-Language-Action)



Adds Language (L) to input. Uses a VLM as backbone with an added Action Head.

Scope of Language Instruction: Guidance vs. Constraints

In Scope: Guidance for Task Completion



- Language provides steps to achieve the goal.
- Helps the agent solve the task.
- Example: "Clean the shoes" -> "Grab rag..."

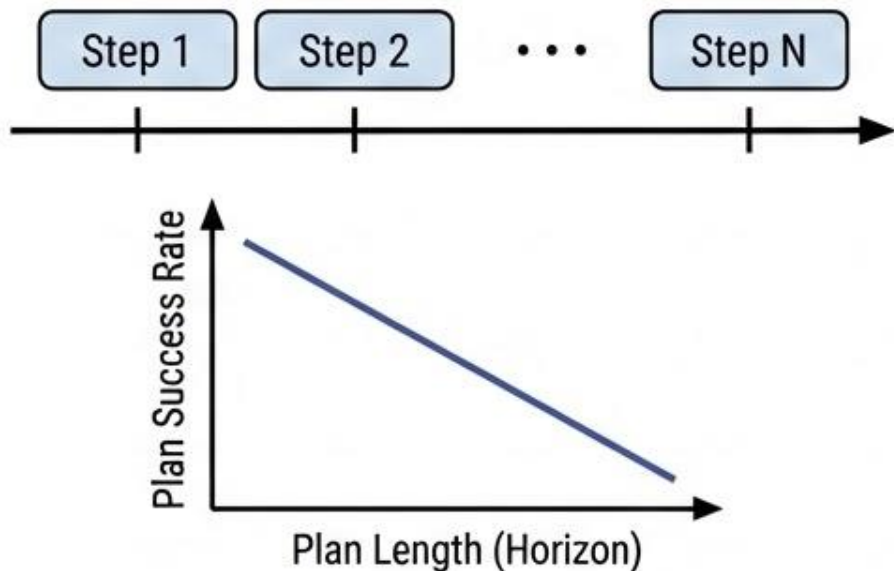
Out of Scope: Additional Constraints



- Language adds restrictions on behavior.
- Changes the fundamental nature of the task.
- Example: "Clean the shoes" -> "Use only one hand..."

Task Difficulty as Planning Demand

Direction 1: Task Horizon (Plan Length)



Success drops as horizon increases (e.g., PlanBench evaluation, LLMs). Long horizon = High difficulty.

Direction 2: Task Width (Concurrent Requirements)



Measures difficulty in terms of parallel or concurrent demands. Width also defines planning demand.

Details in next slide... ➡

Task Difficulty: Horizon vs. Width

Task Horizon: Length $\neq$ Difficulty



- Making 100 sandwiches takes 10x longer than making 10.
- However, cognitive load doesn't increase 10x due to repetition of macro actions.
- Long horizon does not necessarily mean high planning complexity.

Task Width: Concurrency = Difficulty



- Handling 10 balls is significantly harder than handling 2.
- Even if the time duration is the same, the cognitive load is higher due to concurrent tracking.
- High width indicates high planning complexity.

In VLA instruction following, if language already provides decomposed steps, horizon is not an ideal metric for task difficulty.

Instruction Granularity: Guidance and Decomposition

No Consensus Metric

Assemble IKEA Furniture (Complex, Interdependent Task)



Granularity levels
are not comparable
across tasks.

Make Tea (Simple Task, Easy Decomposition)



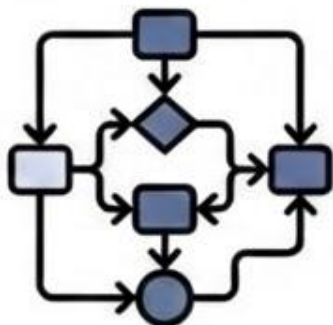
- **Level 2 (Coarse):** “Assemble the bookshelf.”
- **Level 1 (Fine):** “Attach side panel A to base B using screw C.”
- Even “fine” instructions are hard; require context, mistakes are costly.

- **Level 6 (Coarse):** “Make tea.”
- **Level 1 (Fine):** “Boil water.”
- “Fine” instructions are simple, independent, and easy to execute.

Conclusion: Task structure heavily influences decomposition, making it a problematic, non-universal metric.

Intuition for Measuring Granularity: Agent-Centric vs. Task-Centric

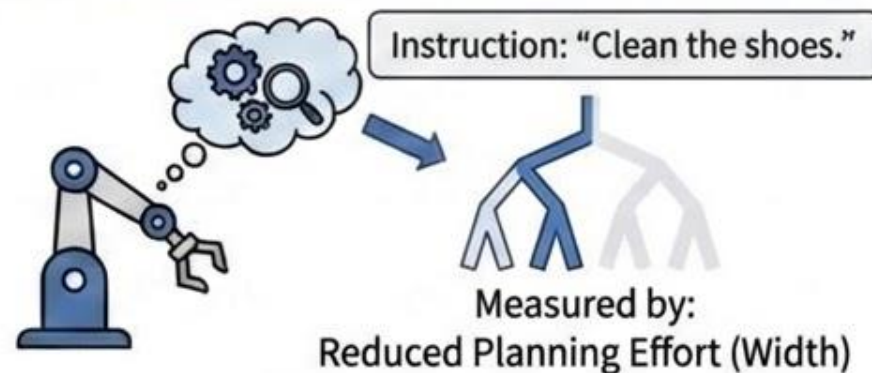
Task-Centric View (Traditional)



Focuses on inherent task structure & complexity.
Independent of the agent executing it.
Examples: Number of subgoals, hierarchy depth, interleaving.

Shift towards
Agent-Centricity

Agent-Centric View (Our Approach)



Focuses on planning demand reduction for the agent.
Measures how much the instruction narrows the search space.
Instruction as guidance to reduce cognitive load.

Core Intuition: Granularity is quantified by the planning effort an agent *saves* due to the instruction's guidance.

Summary & Proposal: Granularity as Planning Width

The Problem with Existing Metrics



Horizon (Length): Ignores cognitive load of concurrency.



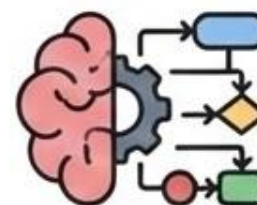
Decomposition (Depth): Task-dependent, fails with interleaved subgoals (Sussman Anomaly).

The Insight: Width = Planning Demand



Concurrency (Width) is a better measure of difficulty than length.
High width = High cognitive load (tracking multiple state variables).

The Proposal: Granularity as Rule Width



Formalize granularity as the *planning demand* imposed by an instruction.

Rule Width: The minimum number of features that must be tracked concurrently.

Quantifiable, architecture-agnostic metric.

Conclusion: Rule Width provides a formal, quantifiable metric for instruction granularity.

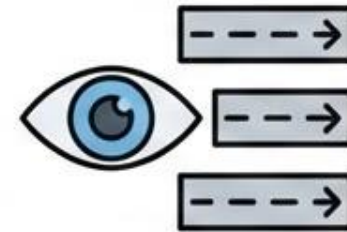
Problem Setting & Informal Definition

Problem Setting: Feature-Based State



We assume a predefined set of domain-level features (Φ) that are crucial for task solving.
These features capture the essential aspects of the world state.

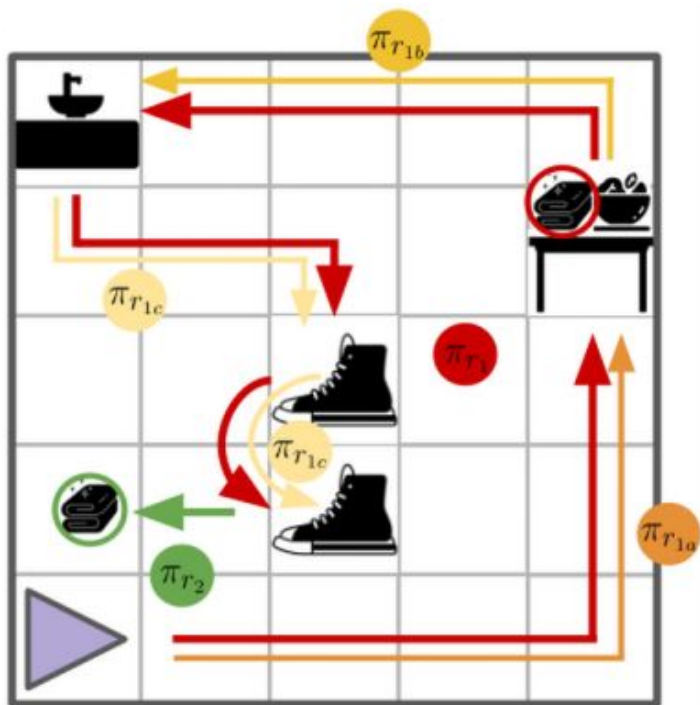
Informal Definition of Width



The 'width' of a task is informally defined as the minimum number of these features that must be tracked concurrently to satisfy the preferred state progression.

Next: A concrete example of how to measure width using the 'cleaning shoes' task.

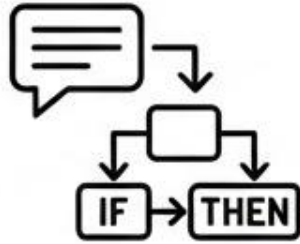
We first illustrate planning width computation using a running example. Table 1 shows the desired state progressions of a task where two shoes are cleaned using a rag, and the rag is placed on the floor afterwards. The key environment dynamics are: (1) the rag must be soaked before it can clean shoes, and (2) the sink must be turned on to soak the rag. We represent states using a set of task-relevant feature variables (Φ): (i) boolean features H_r (holding rag), T_s (sink is turned on), S_r (rag is soaked), C_i (shoe i is cleaned), on_floor_r (rag is on the floor); and (ii) spatial features p_r, p_s, p_f (distance to the rag, sink, and nearest uncleaned shoe)



State Progression	Concurrent Features Tracked (Sequence of Tuples)	Width
Initial State	$[\langle \rangle]$	0
Navigate to rag	$[\langle p_r = X \rangle, \dots, \langle p_r = 0 \rangle]$	1
Pick up rag	$[\langle p_r = 0 \rangle, \langle H_r \rangle]$	1
Navigate to sink	$[\langle H_r, p_s = X \rangle, \dots, \langle H_r, p_s = 0 \rangle]$	2
Turn on sink	$[\langle H_r, p_s = 0 \rangle, \langle T_s \rangle]$	2
Soak rag	$[\langle H_r, T_s, p_s = 0 \rangle, \langle S_r \rangle]$	3
Navigate to Shoe 1	$[\langle H_r, S_r, p_{f1} = X \rangle, \dots, \langle H_r, S_r, p_{f1} = 0 \rangle]$	3
Clean Shoe 1	$[\langle H_r, S_r, p_{f1} = 0 \rangle, \langle C_1 \rangle]$	3
Navigate to Shoe 2	$[\langle H_r, S_r, C_1, p_{f2} \downarrow \rangle, \dots]$	4
Clean Shoe 2	$[\langle H_r, S_r, C_1, p_{f2} = 0 \rangle, \langle C_1, C_2 \rangle]$	4
Put rag on floor	$[\langle C_1, C_2, H_r \rangle, \dots, \langle C_1, C_2, onfloor_r \rangle]$	3

Table 1: State progression and width for the “Cleaning k Shoes” ($k = 2$) w/o instructions. Width scales as $w = k + 2$.

Formal Definition: Instruction Granularity & Width



Symbolic Interpretation: Instruction as Rules

Each instruction is formalized as a set of “if-then” rules $R = \{r_1, \dots, r_m\}$ defined over the feature space Φ . These rules describe the preference of state changes.

- **Example Rule:** “If you hold a rag AND reach a car, THEN you clean the car.”



Formal Definitions: Rule Width & Granularity

Width of a Rule ($w(r)$): The minimum number of features that must be tracked concurrently to achieve the state changes required in the rule.

- **Instruction Granularity:** Defined as the maximum width across its rules:
 $\max(w(r_1), \dots, w(r_m))$.

Next: Applying this formal definition to the ‘cleaning shoes’ task example.

We also append a **counting feature** in Φ to track the number of uncleaned shoes, denoted as N_f

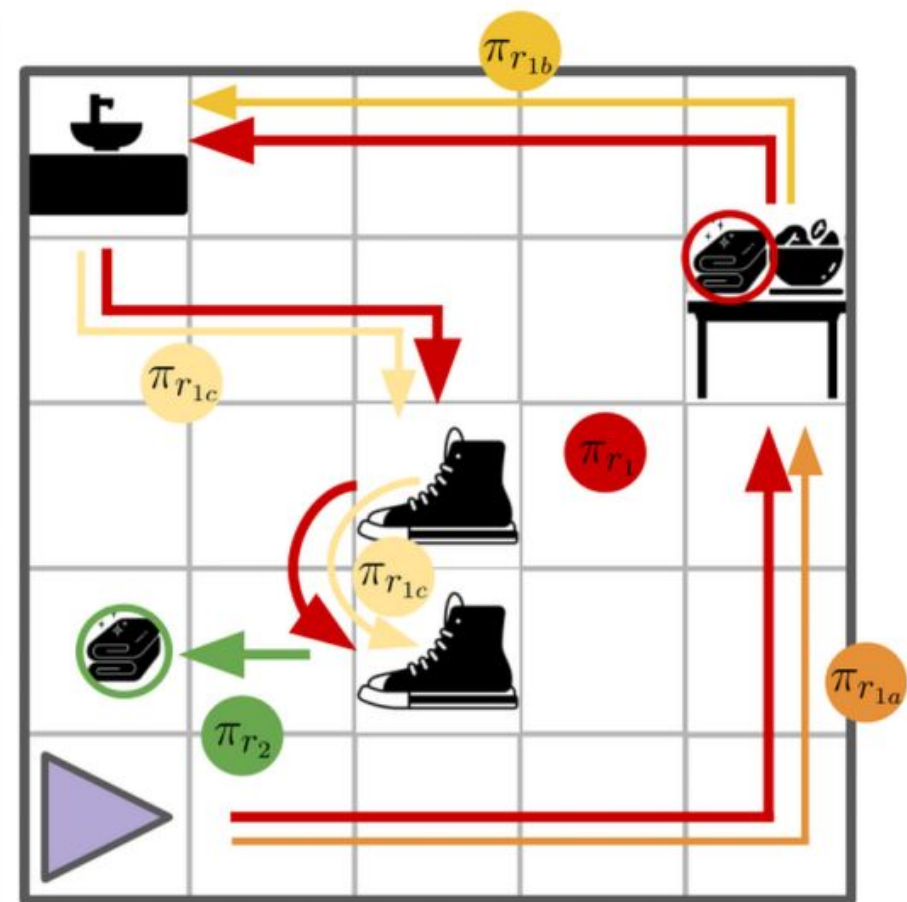
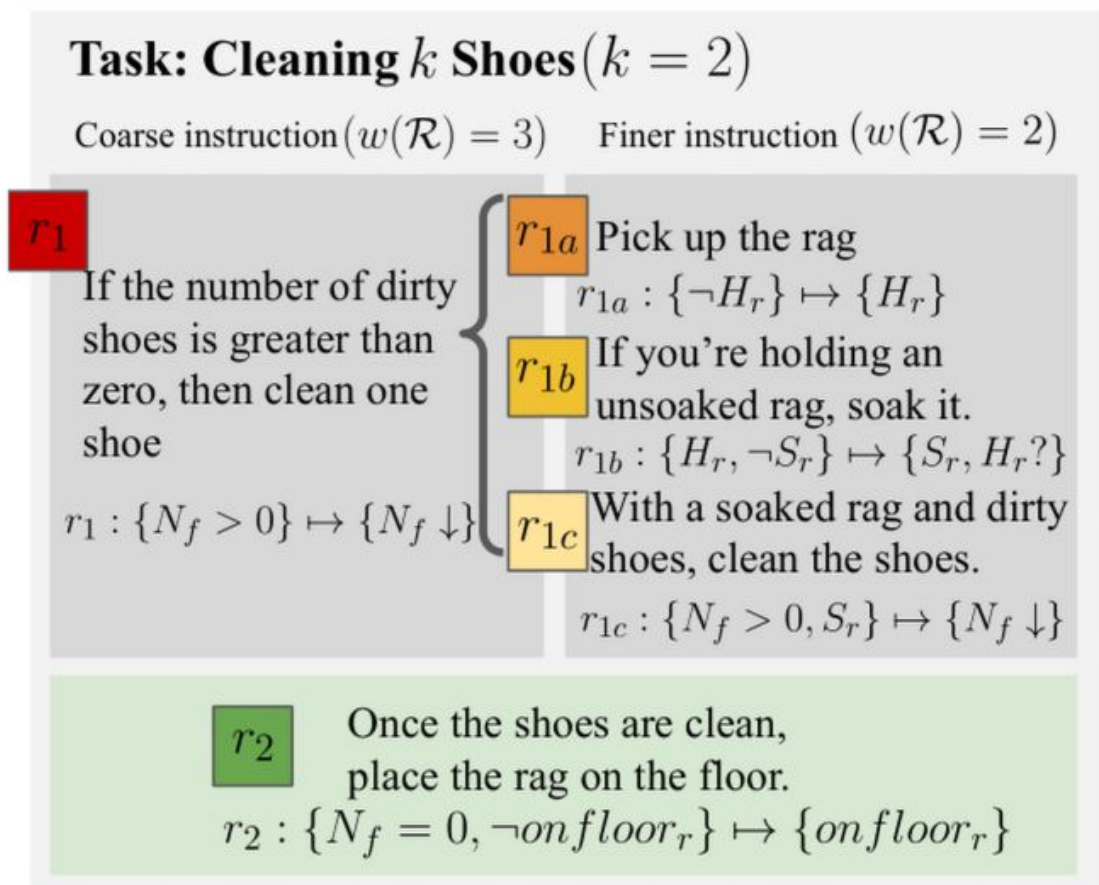


Figure 2: (Left) factoring r_1 into r_{1a} - r_{1c} reduces the planning complexity that the agent must handle internally. More explicit linguistic guidance makes the mapping to action easier to learn. (Right) the long-horizon trajectory for the coarse rule (π_{r_1} , red) is broken into shorter segments by the finer-grained instructions.

State Progression	Concurrent Features Tracked (Sequence of Tuples)	Width
Initial State	$[\langle \rangle]$	0
Navigate to rag	$[\langle p_r = X \rangle, \dots, \langle p_r = 0 \rangle]$	1
Pick up rag	$[\langle p_r = 0 \rangle, \langle H_r \rangle]$	1
Navigate to sink	$[\langle H_r, p_s = X \rangle, \dots, \langle H_r, p_s = 0 \rangle]$	2
Turn on sink	$[\langle H_r, p_s = 0 \rangle, \langle T_s \rangle]$	2
Soak rag	$[\langle H_r, T_s, p_s = 0 \rangle, \langle S_r \rangle]$	3
Navigate to Shoe 1	$[\langle H_r, S_r, p_{f1} = X \rangle, \dots, \langle H_r, S_r, p_{f1} = 0 \rangle]$	3
Clean Shoe 1	$[\langle H_r, S_r, p_{f1} = 0 \rangle, \langle C_1 \rangle]$	3
Navigate to Shoe 2	$[\langle H_r, S_r, C_1, p_{f2} \downarrow \rangle, \dots]$	4
Clean Shoe 2	$[\langle H_r, S_r, C_1, p_{f2} = 0 \rangle, \langle C_1, C_2 \rangle]$	4
Put rag on floor	$[\langle C_1, C_2, H_r \rangle, \dots, \langle C_1, C_2, onfloor_r \rangle]$	3

Middle-Grained Instruction ($w(\mathcal{R}) = 2$) This variant decomposes r_1 to explicitly state the *soaking* process:

- $r_{1a} : \{\neg H_r\} \mapsto \{H_r\}$ (pick up the rag; $w = 1$)
- $r_{1b} : \{H_r, \neg S_r\} \mapsto \{S_r, H_r?\}$ (soak the rag; $w = 2$)
- $r_{1c} : \{N_f > 0, S_r\} \mapsto \{N_f \downarrow\}$ (clean; $w = 2$)
- $r_2 : \{N_f = 0, \neg onfloor_r\} \mapsto \{onfloor_r\}$ (put the rag on the floor; $w = 1$)

Communicating Rules to VLA Agents: Symbolic to Natural Language

Translation Process



To communicate these rules to Vision-Language-Action (VLA) agents, we translate each symbolic rule into a natural-language sentence. These **templates verbalize** the features within each rule.

Example: Rule Translation with Feature Verbalization

Symbolic Rule:

$\{H_r, \neg S_r\} \rightarrow \{S_r\}$

Natural-Language Sentence:

If you are holding the rag but the rag is not yet soaked, soak the rag.

Corresponding segments in symbolic and natural language representations are highlighted with the same color.

Task	Split	Class	Goal Description	Mean Steps
opening_packages	ID	short	Open every package.	10.2
installing_a_printer	ID	short	Place a printer on a table and turn it on.	10.6
laying_wood_floors	ID	short	Lay every plywood sheet on the floor, each touching at least one other sheet.	15.9
moving_boxes_to_storage	ID	short	Place every carton inside a shelf.	36.3
cleaning_a_car	ID	short	Make at least one car dust-free. Place a rag and soap together inside a bucket.	62.6
throwing_away_leftovers	ID	middle	Throw away every hamburger by placing it in a trash can.	68.3
organizing_file_cabinet	ID	middle	Put a marker on a table and file every folder and document in a cabinet.	75.5
making_tea	ID	middle	Slice a lemon. Put a teapot on an activated stove. Keep a soaked teabag with the teapot.	78.7
sorting_books	ID	middle	Place every book and every hardback on a shelf.	78.8
collect_misplaced_items	ID	middle	Place all gym shoes, necklaces, notebooks, and socks on a table.	90.7
thawing_frozen_food	ID	middle	Place a date label next to a fish. Put every fish next to a sink. Set an olive next to a sink.	101.7
boxing_books_up_for_storage	ID	long	Place every book inside a box.	134.4
storing_food	ID	long	Store every cookable item inside a cabinet.	144.6
preparing_salad	ID	long	Slice all apples and tomatoes; place them on plates. Put every radish and lettuce on plates. Ensure each plate holds at least one cookable item.	146.7
putting_away_dishes_after_cleaning	ID	long	Put every plate inside a cabinet.	147.7
cleaning_up_the_kitchen_only	ID	long	Put every blender on a countertop. Refrigerate all apples and casseroles. Keep soap next to a sink. Make plates unstained and cabinets dust-free. Place each rag either beside or inside the sink. Store plates and vegetable oil in different cabinets.	161.0
setting_up_candles	ID	long	On every table, place three distinct candles on top.	165.2
cleaning_shoes	OOD	short	Leave a towel on the floor and make sure every shoe is unstained and dust-free.	54.2
watering_houseplants	OOD	middle	Water every potted plant until it is soaked.	79.4
washing_pots_and_pans	OOD	long	Clean every teapot, kettle, and pan so they're unstained, then store each inside a cabinet.	149.5

Table 6: Classification of tasks in MINI-BEHAVIOR-GRAN based on the median number of plan steps, as determined by the BWFS planner. Tasks are categorized into short, middle, and long classes to reflect varying planning complexity based on plan length.

Task	Split	Max Width	Relative Granularity Bins		
			Fine	Medium	Coarse
opening_packages	ID	1	0	0	1
installing_a_printer	ID	2	0	1	2
laying_wood_floors	ID	2	0	1	2
moving_boxes_to_storage	ID	2	0	1	2
cleaning_a_car	ID	3	0	1	2,3
throwing_away_leftovers	ID	2	0	1	2
organizing_file_cabinet	ID	3	0	1	2,3
making_tea	ID	3	0	1	2,3
sorting_books	ID	2	0	1	2
collect_misplaced_items	ID	2	0	1	2
thawing_frozen_food	ID	2	0	1	2
boxing_books_up_for_storage	ID	2	0	1	2
storing_food	ID	3	0	1	2,3
preparing_salad	ID	4	0	1,2	3,4
putting_away_dishes_after_cleaning	ID	3	0	1	2,3
cleaning_up_the_kitchen_only	ID	5	0,1	2,3	4,5
setting_up_candles	ID	2	0	1	2
cleaning_shoes	OOD	3	0	1	2,3
watering_houseplants	OOD	3	0	1	2,3
washing_pots_and_pans	OOD	4	0	1,2	3,4

Table 7: Per-task instruction width ranges and relative granularity binning in MINI-BEHAVIOR-GRAN. Symbolic widths are normalized into Fine (Low), Medium, and Coarse (High) levels using a task-specific quantile-based strategy.

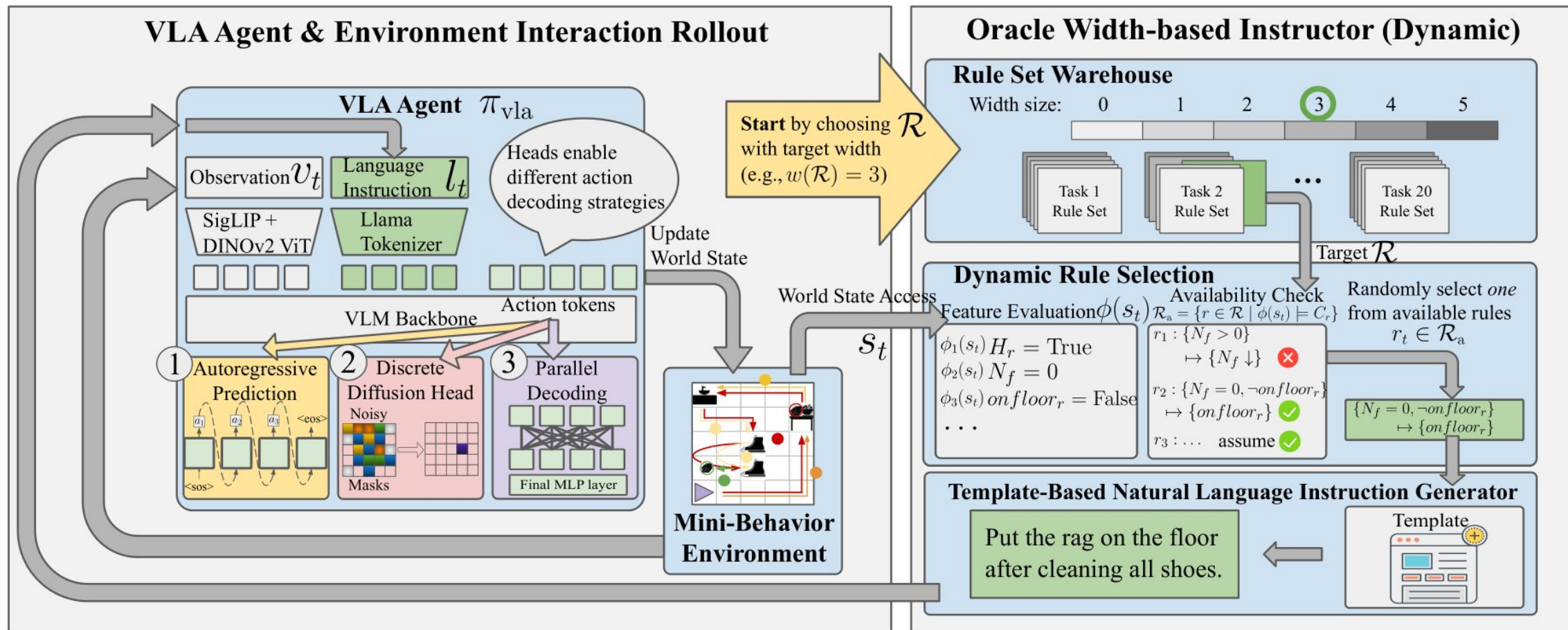


Figure 1: MINI-BEHAVIOR-GRAN Experimental Pipeline. We evaluate the impact of instruction granularity across three VLA action-decoding strategies. During inference time, an Oracle Instructor evaluates the current world state to decide available rule set $\mathcal{R}_a$ and active rule r_t . A rule-based natural language instruction l_t is generated accordingly. The process iterates until completion or max steps.

Research Questions. We address the following questions:

(Q1) Granularity Trade-offs. How does instruction granularity impact Success Rate (**SR**) on within splits?

(Q2) Generalization. Can VLA agents trained on certain granularities generalize to other unseen granularities?

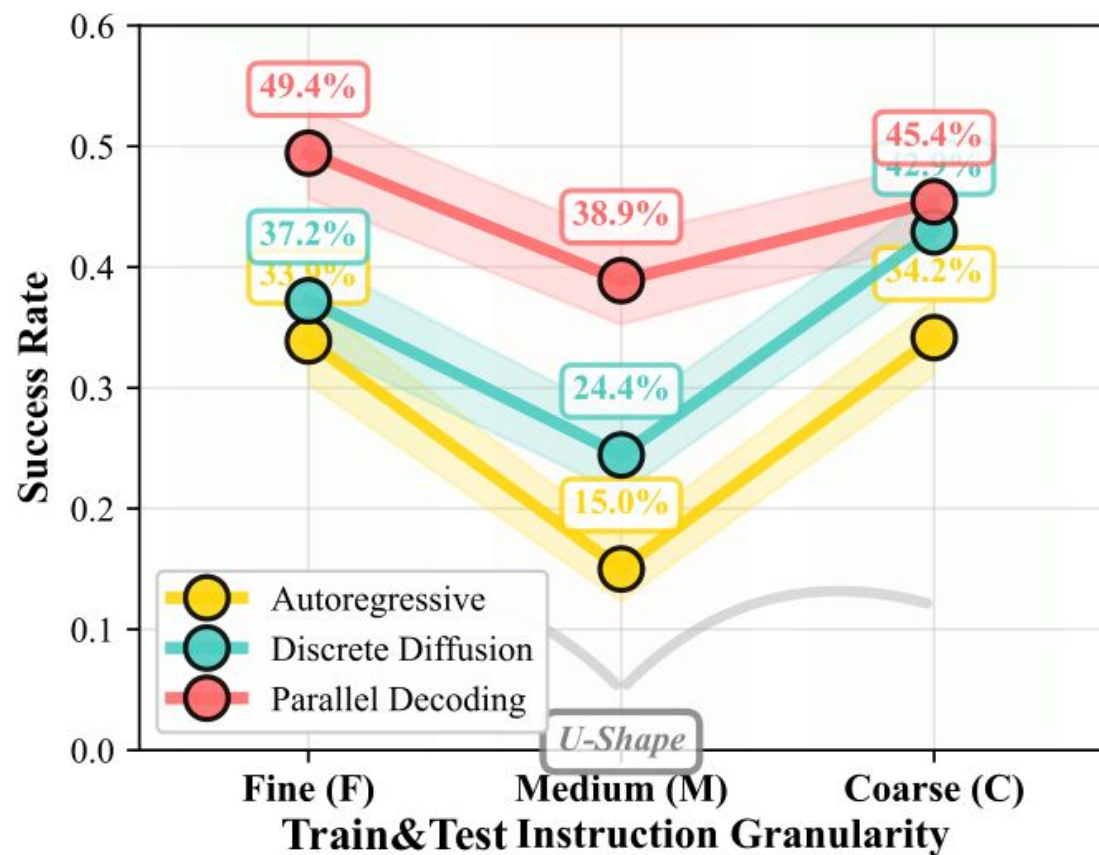


Figure 3: In-Distribution (matched train/test) SR across granularities (i.e., train Fine→test Fine, etc). A U-shaped trend appears.

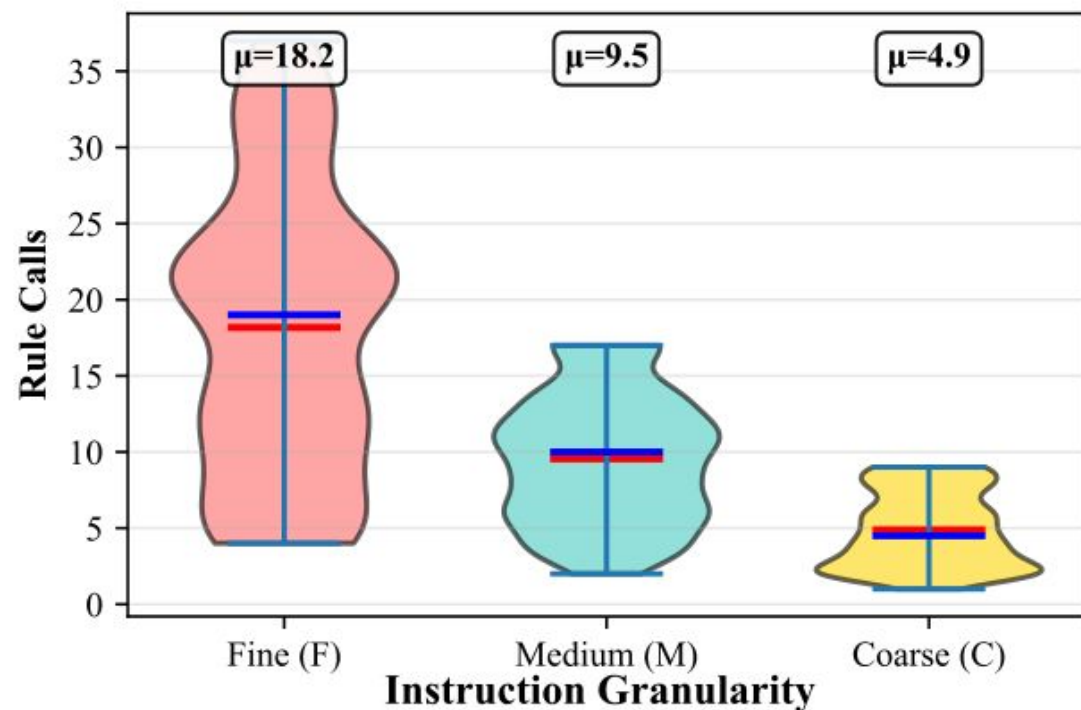


Figure 5: Rule call distribution across instruction granularities. # of rule calls decreases with coarser instructions. It decouples instruction length from width.

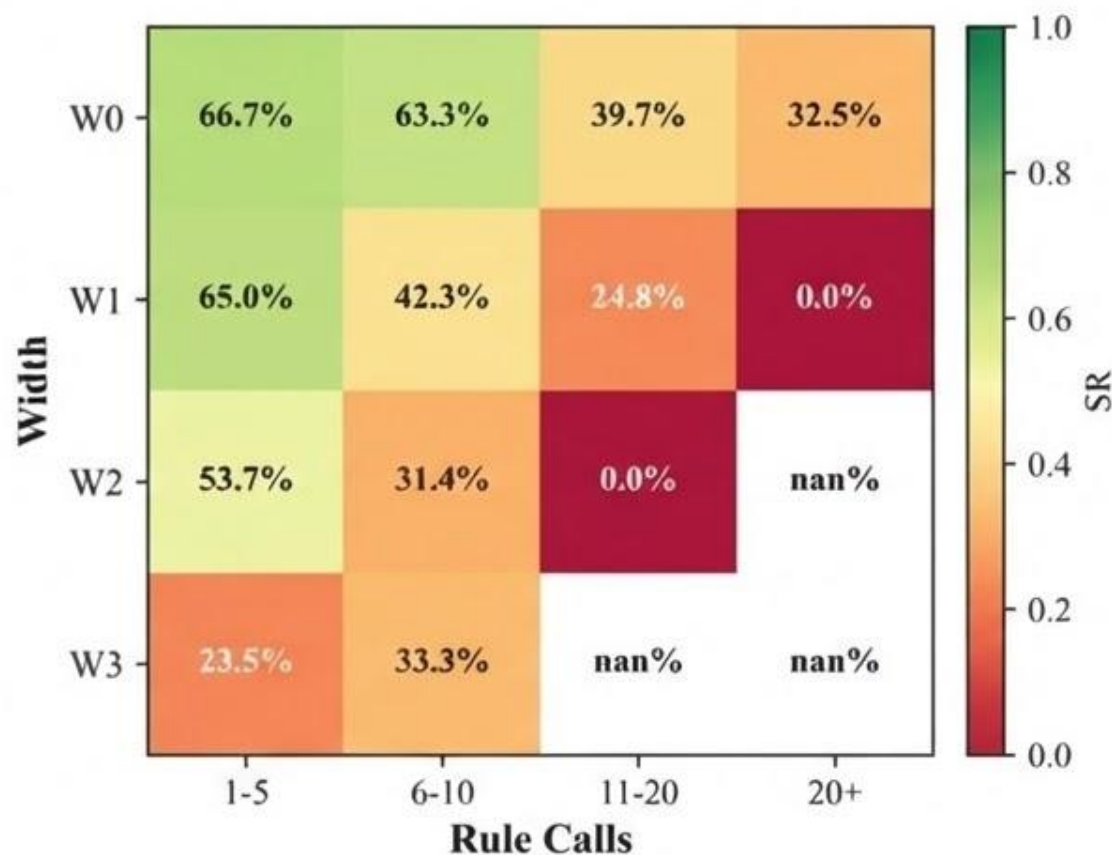


Figure 6: Impact of width and horizon on SR. SR decays with task horizon; larger instruction width imposes an additional burden even when horizon is fixed.

U-Shaped Discovery in Instruction Width

- **Non-monotonic U-shape:** Observed across all action-decoding archetypes.
- **Controlled Generation:** Template-based translator isolates **rule width** as the causal factor (uniform vocabulary, syntax).
- **Fine ($w=0$):** Minimal learning burden. Direct step-by-step mapping requires little reasoning.
- **Coarse ($w \geq 2$):** Fosters internal planning. Broader patterns enable robust, generalizable language grounding.
- **Medium ($w=1$):** Sub-optimal grounding regime. Too sparse for direct mapping, not coarse enough for planning, leading to performance decline.

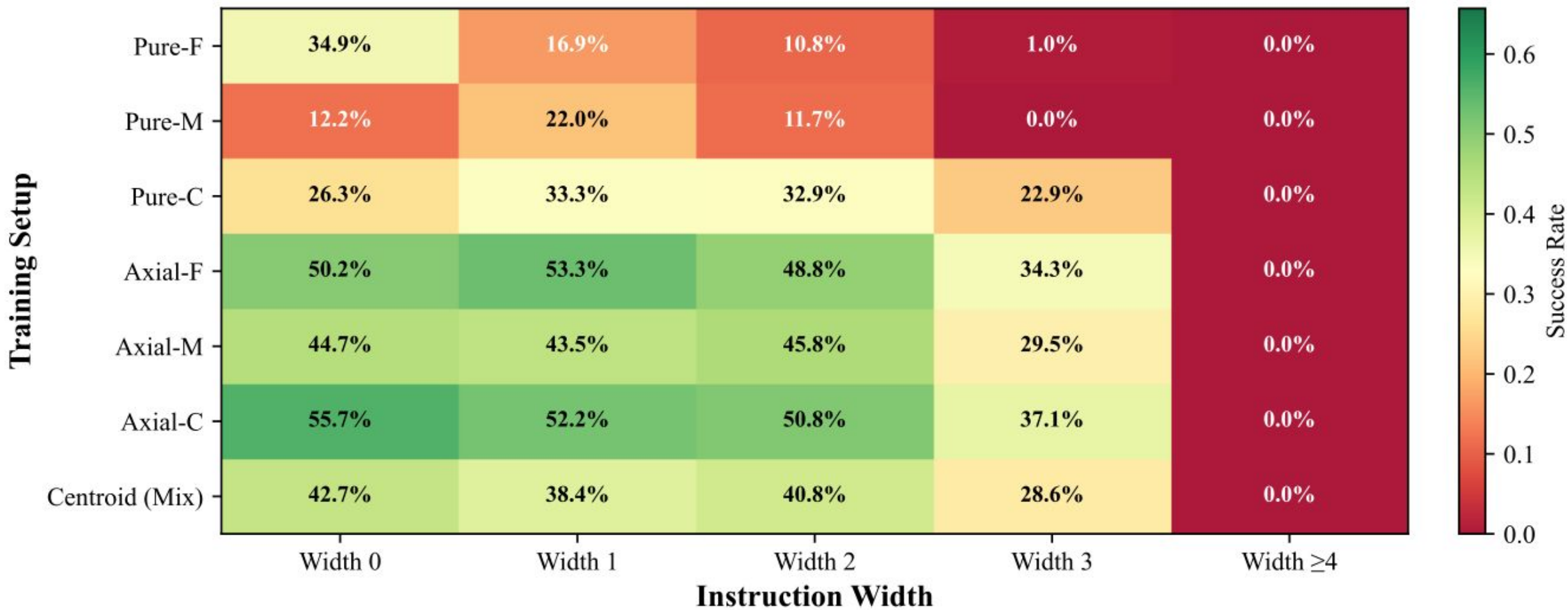


Figure 4: The x-axis are test instruction width, and the y-axis are training setup. Transfer is asymmetric: train coarse $\rightarrow$ test fine works, but train fine $\rightarrow$ test coarse does not. Mixed-granularity training improves generalization. *Axial-C* training setting performs best overall.

I would like to stop here.
Thanks for listening!
Feel free to ask anything!